



GitLab CE 19.2 完整安装、使用与管理员实战指南

版本支持：GitLab Community Edition (CE) 19.2.1 适用环境：Ubuntu 20.04/22.04/24.04 LTS、Debian 11/12、CentOS 7/8/9、Rocky Linux、Windows WSL2 / Docker Desktop 维护者：sheng13 社区教程

目录

- [一、前言与硬件/环境要求](#)
- [二、GitLab 整体系统架构图解](#)
- [\[三、部署方案一：基于 Docker Compose 本地一键部署（强烈推荐）\]\(#三-部署方案一基于-docker-compose-本地一键部署强烈推荐\)](#)
- [四、部署方案二：Linux 原生包安装 \(Omnibus Package\).](#)
- [五、首次登录与系统初始化安全配置](#)
- [六、GitLab 核心使用指南与权限管理](#)
- [七、GitLab CI/CD 自动化流水线实战](#)
- [八、数据备份、恢复与定时维护](#)
- [九、常见故障排查 \(Troubleshooting FAQ\).](#)
- [十、GitHub 仓库提交与版本管理指南](#)

一、前言与硬件/环境要求

GitLab 是目前业界使用最广泛的开源 DevOps 平台，集成了代码托管、Issue 追踪、Merge Request 代码审查、CI/CD 自动化持续集成与部署、Package 依赖包管理以及安全扫描等全套功能。

1.1 硬件配置推荐

GitLab 是基于 Ruby on Rails、Go、Vue.js 及 PostgreSQL 构建的大型应用，在本地部署时对内存与 CPU 资源有一定要求：

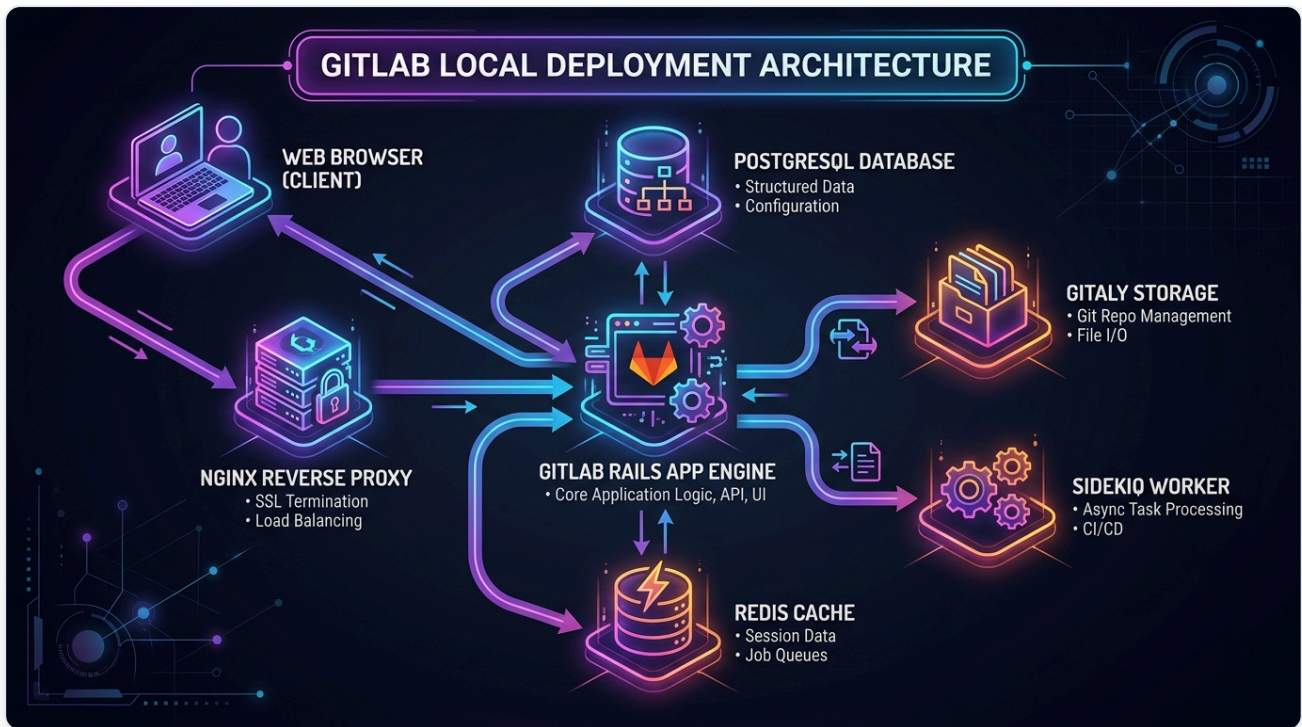
评估维度	最低配置 (极简体验)	推荐配置 (小团队 1-20人)	生产推荐 (20-100人)
CPU 核心数	2 核 (Dual-Core)	4 核 (Quad-Core)	8 核以上
内存 (RAM)	4 GB (需配置 4GB SWAP)	8 GB RAM	16 GB RAM 以上
磁盘空间	20 GB NVMe/SSD	100 GB+ SSD	500 GB+ 高速 SSD
操作系统	Linux (Ubuntu/Debian/CentOS) 或 Windows 11 (WSL2 Docker)		

 **关键建议：**如果服务器只有 4GB 内存，**必须手动开启至少 4GB 的 SWAP 交换内存**，并在 `gitlab.rb` 中限制 Puma 进程数和 PostgreSQL 缓冲区，否则启动时极易发生 502 错误或内存溢出 (OOM Killer)。

二、GitLab 整体系统架构图解

下图展示了 GitLab 本地私有化部署的核心架构，包括前端反向代理、应用核心组件、数据存储与后台任务调度协同关系：

！



GitLab 系统架构图

核心组件职责解析：

1. **Nginx**：作为前端入口反向代理，处理 HTTP/HTTPS 请求与 SSL 证书解密。
2. **Puma**：GitLab 核心 Web 应用服务器（基于 Ruby on Rails），处理业务逻辑与页面渲染。
3. **Workhorse**：高性能 Go 语言智能反向代理，处理大文件上传/下载、Git HTTP 操作与 Websocket。
4. **Gitaly**：GitLab 专属的 Go 语言 RPC 服务，高效处理所有 Git 磁盘读写与 Repository 操作。
5. **PostgreSQL**：元数据库，存储用户、项目信息、Issue、Merge Request 等元数据。
6. **Redis**：缓存系统与 Sidekiq 任务队列底座。
7. **Sidekiq**：异步后台任务执行器（如邮件发送、CI/CD 调度、代码统计等）。

三、部署方案一：基于 Docker Compose 本地一键部署（强烈推荐）

使用 Docker Compose 是最推荐的部署方式，具有环境隔离、升级便捷、配置解耦等极大优势。

3.1 部署架构与网络拓扑

！

Docker Compose 部署拓扑图

3.2 步骤 1: 创建本地存储目录

在宿主机上创建持久化数据目录，确保容器毁弃重建后数据不丢失：

```
sudo mkdir -p /srv/gitlab/config
sudo mkdir -p /srv/gitlab/logs
sudo mkdir -p /srv/gitlab/data
```

3.3 步骤 2: 编写 `docker-compose.yml` 生产级配置文件

在 `/srv/gitlab` 目录下创建 `docker-compose.yml` 文件:

```

version: '3.8'

services:
  gitlab:
    image: 'gitlab/gitlab-ce:19.2.1-ce.0'
    container_name: 'gitlab'
    restart: always
    hostname: 'gitlab.local'
    environment:
      GITLAB_OMNIBUS_CONFIG: |
        # 外部访问 URL（替换为您的宿主机实际 IP 或域名）
        external_url 'http://192.168.70.196:8080'
        nginx['listen_port'] = 80

        # 修改 GitLab 内置 SSH 端口以避免与宿主机 22 端口冲突
        gitlab_rails['gitlab_shell_ssh_port'] = 8022

        # 时区设置
        gitlab_rails['time_zone'] = 'Asia/Shanghai'

        # -----
        # 内存性能优化配置（适合 4GB ~ 8GB 内存的环境）
        # -----
        puma['worker_processes'] = 2
        puma['per_worker_max_memory_mb'] = 1024
        sidekiq['max_concurrency'] = 10
        postgresql['shared_buffers'] = "256MB"
        prometheus_monitoring['enable'] = false

    ports:
      - '8080:80'      # Web HTTP 访问端口
      - '8443:443'     # HTTPS 端口（如启用 SSL）
      - '8022:22'      # Git SSH 提交端口
    volumes:
      - '/srv/gitlab/config:/etc/gitlab'
      - '/srv/gitlab/logs:/var/log/gitlab'
      - '/srv/gitlab/data:/var/opt/gitlab'
    shm_size: '256m'

```

3.4 步骤 3：启动容器并查看日志

执行以下命令启动 GitLab 服务：

```
cd /srv/gitlab
# 启动容器
docker compose up -d

# 实时查看启动日志（首次启动需要 2 ~ 5 分钟进行数据库初始化）
docker logs -f gitlab
```

四、部署方案二：Linux 原生包安装 (Omnibus Package)

如果您希望直接在 Linux 物理机或虚拟机上原生运行，可以使用 GitLab Omnibus 官方/镜像源安装。

4.1 安装基础依赖 (以 Ubuntu/Debian 为例)

```
sudo apt-get update
sudo apt-get install -y curl openssh-server ca-certificates tzdata perl
```

4.2 配置清华大学开源软件镜像源 (加速下载)

创建并修改包源文件 `/etc/apt/sources.list.d/gitlab.list`：

```
curl -fsSL https://packages.gitlab.com/gpg.key | sudo gpg --dearmor -o /etc/apt/trusted.g
echo "deb https://mirrors.tuna.tsinghua.edu.cn/gitlab/ubuntu jammy main" | sudo tee /etc/
```

4.3 执行安装与重配置

```
sudo apt-get update
# 安装 GitLab CE 最新版本
sudo EXTERNAL_URL="http://192.168.70.196" apt-get install gitlab

# 首次重配置与初始化（生成配置文件并启动所有服务）
sudo gitlab-ctl reconfigure
```

4.4 常用 `gitlab-ctl` 管理命令

- 启动服务： `sudo gitlab-ctl start`
- 停止服务： `sudo gitlab-ctl stop`
- 重启服务： `sudo gitlab-ctl restart`
- 查看状态： `sudo gitlab-ctl status`
- 实时日志： `sudo gitlab-ctl tail`

五、首次登录与系统初始化安全配置

!

The image shows the GitLab Community Edition login page. At the top, it says "GitLab Community Edition". Below that is a form with two input fields: "Username or primary email" and "Password". The "Password" field has a toggle icon (an eye) to the right. Below the "Password" field is a link that says "Forgot your password?". There is a checkbox labeled "Remember me" below the "Forgot your password?" link. Below the checkbox are two buttons: a dark "Sign in" button and a light "Passkey" button with a key icon. At the bottom of the page, there are links for "Explore", "Help", "About GitLab", and "GitLab community forum". On the right side of the bottom, there is a language selector that says "English" with a dropdown arrow.

GitLab 19.2 登录页面

5.1 获取管理员 `root` 默认密码

GitLab 在安装完成后会自动生成一个随机初始密码，有效期为 24 小时：

```
# Docker 部署方式获取：
docker exec -it gitlab cat /etc/gitlab/initial_root_password

# 原生安装方式获取：
sudo cat /etc/gitlab/initial_root_password
```

5.2 登录界面与修改 Root 密码

1. 在浏览器中打开：`http://<您的服务器IP>:8080`
2. 用户名输入 `root`，密码输入上一步获取到的字符串。
3. 登录后立即进入 **Admin Area (管理员区域)** -> **Users** -> **Edit Root**，重置密码为强密码并保存。

5.3 开启中文语言界面

1. 点击右上角用户头像 -> **Preferences (首选项)**。
2. 滚动找到 **Localization (本地化)** -> **Language (语言)**。
3. 选择 **简体中文**，点击 **Save changes**，刷新页面即可呈现全中文界面。

5.4 安全加固与 SMTP 发信设置

修改配置文件 `/srv/gitlab/config/gitlab.rb` 或 Docker 中的 `GITLAB_OMNIBUS_CONFIG`，添加 SMTP 邮箱发信支持（以 QQ 企业邮箱/网易邮箱为例）：

```
gitlab_rails['smtp_enable'] = true
gitlab_rails['smtp_address'] = "smtp.exmail.qq.com"
gitlab_rails['smtp_port'] = 465
gitlab_rails['smtp_user_name'] = "notification@yourdomain.com"
gitlab_rails['smtp_password'] = "YourAuthorizedPassword"
gitlab_rails['smtp_authentication'] = "login"
gitlab_rails['smtp_enable_starttls_auto'] = false
gitlab_rails['smtp_tls'] = true
gitlab_rails['gitlab_email_from'] = 'notification@yourdomain.com'
```

保存后重新运行：`docker exec -it gitlab gitlab-ctl reconfigure`。

六、GitLab 核心使用指南与权限管理

6.1 GitLab 组织架构与 5 大角色权限

GitLab 遵循 **Group (项目组)** -> **Sub-group (子组)** -> **Project (项目)** 的树状结构。

```
🏢 公司/团队 (Group)
├── 📁 前端团队 (Sub-group)
│   ├── 📁 Vue3-App (Project)
│   └── 📁 Design-System (Project)
└── 📁 后端团队 (Sub-group)
    └── 📁 Go-Microservice (Project)
```

成员角色权限对比：

角色 (Role)	代码 Clone/Pull	代码 Push	创建分支/MR	修改项目设置	管理组员与权限
Guest (访客)	✗ (受限)	✗	✗	✗	✗
Reporter (报告者)	选定项目	✗	✗	✗	✗
Developer (开发者)	✓	✓ (未受保护分支)	✓	✗	✗
Maintainer (维护者)	✓	✓ (所有分支)	✓	✓	✗
Owner (所有者)	✓	✓	✓	✓	✓

6.2 本地配置 SSH Key 密钥

在本地开发机终端中生成 SSH 密钥并添加到 GitLab，以实现免密 Pull/Push：

```
# 1. 生成 Ed25519 密钥
ssh-keygen -t ed25519 -C "your_email@example.com"

# 2. 查看并复制公钥内容
cat ~/.ssh/id_ed25519.pub

# 3. 在 GitLab Web 界面中操作：
# 点击右上角头像 -> 编辑个人资料 -> SSH 密钥 -> 添加新密钥 -> 粘贴并保存。
```

6.3 保护分支 (Protected Branches) 与 Code Review 流水线

- 进入项目 **设置 (Settings)** -> **仓库 (Repository)** -> **受保护分支 (Protected branches)**。
- 将 `main` 或 `master` 分支设置为：
 - Allowed to push:** `No one` (禁止任何人直接 Push 代码)
 - Allowed to merge:** `Maintainers` (仅允许维护者审核 Merge Request 后合并)
- 开发者标准工作流：
 - 本地创建特性分支：`git checkout -b feature/login-page`
 - 提交代码并推送：`git push origin feature/login-page`
 - 在 GitLab 界面发起 **Merge Request (MR)**，等待 Leader 审查合并。

管理员 Admin Area 完整操作

管理员从左上角菜单进入 **Admin**。这里管理整个实例，不是单一项目。

用户生命周期

在 **Admin → Overview → Users** 可建立、封锁、停用或删除用户。离职账号应先 Block，转移群组 and 项目所有权、撤销 Token 与 SSH Key，再按保留政策处理；不要一开始就删除。**Admin** 权限只给极少数可信人员。Impersonate 仅用于排障，结束后立即退出并保留审计记录。

群组、项目与权限

用顶层 Group 表示组织，再以 Subgroup 区部门或产品，让项目继承群组成员。普通开发者给 Developer，负责人给 Maintainer，Owner 仅给群组负责人。项目默认 Private，并保护 **main**：禁止直接 push，要求 Merge Request、成功流水线和审查。

注册、可见性与系统设置

在 **Admin → Settings → General** 关闭公开注册，限制 Public/Internal 项目创建，设置默认项目可见性和新用户权限。保存后用普通账号或无痕窗口验证。

安全与凭证

管理员和 root 必须使用强密码及 2FA，恢复代码离线保存。个人访问 Token 只给必要 scope 并设置到期日；CI 秘密放在 masked/protected Variables，不写进仓库、URL 或日志。定期检查管理员、长期未用账号、过期 Token 与异常 Runner。

SMTP、限额与维护

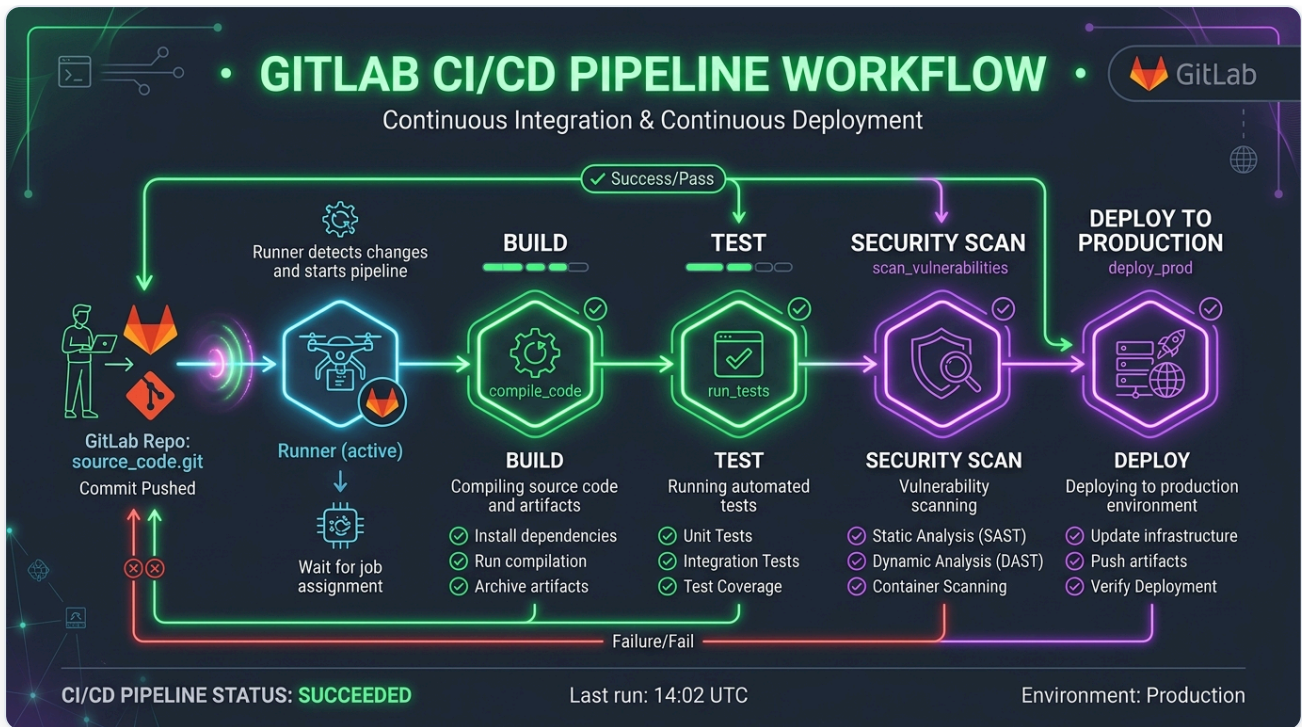
配置 SMTP 后执行 `docker exec gitlab gitlab-ctl reconfigure` 并发送测试邮件。规划 Repository、LFS、Artifacts 与 Registry 限额，流水线产物设置 `expire_in`。管理员每周检查磁盘、Sidekiq、Runner、备份新鲜度与证书到期日。

七、GitLab CI/CD 自动化流水线实战

GitLab 内置了强大的 CI/CD 引擎，只需在项目根目录创建 `.gitlab-ci.yml` 即可触发自动化构建与部署。

7.1 CI/CD 流水线工作原理图

！



GitLab CI/CD 工作流

7.2 步骤 1：本地部署并注册 GitLab Runner

GitLab Runner 是用于真正执行 CI/CD 任务的具体构建节点。

1. 运行 Docker 版本的 GitLab Runner

```
docker run -d --name gitlab-runner --restart always \
  -v /srv/gitlab-runner/config:/etc/gitlab-runner \
  -v /var/run/docker.sock:/var/run/docker.sock \
  gitlab/gitlab-runner:alpine-v19.2.1
```

2. 先在项目/群组 Runner 页面建立 Runner，再使用短期显示的 authentication token 注册

```
docker exec -it gitlab-runner gitlab-runner register \
  --non-interactive \
  --url "http://192.168.70.196:8080/" \
  --token "glrt-REDACTED" \
  --executor "docker" \
  --docker-image "node:18-alpine" \
  --description "Local-Docker-Runner" \
  --tag-list "docker,build"
```

7.3 步骤 2：编写 .gitlab-ci.yml 管道脚本

在代码项目根目录下创建 `.gitlab-ci.yml`：

```
stages:
  - install
  - test
  - build
  - deploy

cache:
  paths:
    - node_modules/

install_deps:
  stage: install
  script:
    - npm install
  tags:
    - docker

run_tests:
  stage: test
  script:
    - npm run test
  tags:
    - docker

build_artifact:
  stage: build
  script:
    - npm run build
  artifacts:
    paths:
      - dist/
  tags:
    - docker

deploy_prod:
  stage: deploy
  script:
    - echo "Deploying build artifacts to production server..."
  only:
    - main
  tags:
    - docker
```

八、数据备份、恢复与定时维护

数据安全是私有部署的重中之重。

8.1 一键执行备份

```
# Docker 容器执行备份命令：
docker exec -t gitlab gitlab-backup create

# 原生安装备份命令：
sudo gitlab-backup create
```

默认备份文件会保存在 `/var/opt/gitlab/backups`（挂载在宿主机的 `/srv/gitlab/data/backups`）目录下，文件名形如：`1786350457_2026_08_10_19.2.0_gitlab_backup.tar`。

⚠ 完整备份有三层：应用备份 tar；`/etc/gitlab` 内的 `gitlab.rb`、`gitlab-secrets.json`、证书和 SSH 主机密钥；镜像精确版本、Compose、端口、卷路径和 SHA-256 清单。缺少 secrets 可能无法解密 CI 变量与 Token。备份必须另存到另一台机器或加密存储。

8.2 设置 Linux Cron 定时自动备份脚本

在宿主机上创建自动备份脚本 `/usr/local/bin/gitlab_auto_backup.sh`：

```
#!/bin/bash
# 1. 执行备份
docker exec -t gitlab gitlab-backup create

# 2. 只有在备份成功后才生成校验清单；删除策略应另行审核
sha256sum /srv/gitlab/data/backups/*_gitlab_backup.tar > /srv/gitlab/SHA256SUMS
# 3. 另行复制 /etc/gitlab、Compose、备份与清单到异机存储
```

给予执行权限并添加到系统的 crontab：

```
chmod +x /usr/local/bin/gitlab_auto_backup.sh
# 每日凌晨 2:00 自动执行
(crontab -l 2>/dev/null; echo "0 2 * * * /usr/local/bin/gitlab_auto_backup.sh") | crontab
```

8.3 数据灾难恢复全流程

如遇物理机损坏，需恢复数据到新机器：

```
# 1. 恢复 Secrets 配置文件
cp gitlab-secrets.json /srv/gitlab/config/

# 2. 将备份 tar 包放入 backups 目录
cp 1786350457_2026_08_10_19.2.0_gitlab_backup.tar /srv/gitlab/data/backups/

# 3. 停止数据库写入服务
docker exec -it gitlab gitlab-ctl stop puma
docker exec -it gitlab gitlab-ctl stop sidekiq

# 4. 执行数据恢复（注意指定时间戳前缀）
docker exec -it gitlab gitlab-backup restore BACKUP=1786350457_2026_08_10_19.2.0

# 5. 重启并校验
docker exec -it gitlab gitlab-ctl restart
```

九、常见故障排查 (Troubleshooting FAQ)

Q1: 访问页面出现 502 Bad Gateway?

- **原因 1:** 物理内存不足, Puma 进程被操作系统 OOM Killer 杀掉。 **解决:** 增加 4GB SWAP 交换内存, 并在 `gitlab.rb` 中缩减 `puma['worker_processes'] = 2`。
- **原因 2:** 后台服务正在初始化中 (需等待 2-5 分钟)。 **解决:** 运行 `docker exec -it gitlab gitlab-ctl status` 检查是否有服务一直在 `down` 状态。

Q2: 使用 SSH 克隆提示 `Permission denied (publickey)` ?

- **原因:** 本地端口未映射正确或 GitLab 内置 SSH 端口配置不匹配。 **解决:** 检查克隆地址是否带端口, 如: `git clone ssh://git@192.168.70.196:8022/group/project.git`。

Q3: 如何修改已部署 GitLab 的 IP 或域名?

- **操作:** 修改 `/srv/gitlab/config/gitlab.rb` 中的 `external_url 'http://new-ip:8080'`, 然后运行 `docker exec -it gitlab gitlab-ctl reconfigure`。

升级、安全验收与本机实测

升级前五项检查

1. 阅读官方 release notes 与 upgrade path, 跨版本时逐个经过 required upgrade stops。

2. 记录当前精确版本、edition、镜像 digest、端口与卷路径。
3. 生成应用备份，并另存 `/etc/gitlab` 、Compose 与校验清单。
4. 检查磁盘空间、后台迁移和当前容器健康度。
5. 事先写好回退方案；数据库迁移后不能只靠换回旧镜像回退。

将 Compose 镜像从 `19.2.0-ce.0` 固定改为 `19.2.1-ce.0` 后：

```
docker compose pull gitlab
docker compose up -d gitlab
docker logs -f gitlab
```

初始化时短暂 502 常见。看到 `gitlab-ctl reconfigure` 仍在运行时应等待，不要连续重启。完成后检查：

```
docker inspect -f '{{.Config.Image}} {{.State.Health.Status}}' gitlab
docker exec gitlab gitlab-ctl status
curl -I http://192.168.70.196:8080/users/sign_in
ssh -T -p 8022 git.168.70.196
```

还要人工测试登录、项目、仓库 clone/push 、Issue 、MR、流水线、Runner、邮件与新备份。`healthy` 不等于所有业务验收完成。

2026-08-10 本机结果

检查	结果
版本	GitLab CE 19.2.1
Web	<code>http://192.168.70.196:8080</code> , HTTP 200
SSH	宿主机 8022 转发容器 22
容器	healthy, 核心服务为 run
数据	继续使用原 config/logs/data 卷
升级前备份	<code>1786350457_2026_08_10_19.2.0_gitlab_backup.tar</code>
SHA-256	<code>0fd492a5cc715e026601097065b2817aa644bb19e676b1738e25d669d194b852</code>

配置密钥保存在受限独立目录，不提交至公开仓库。

Runner 特别安全提醒

把 `/var/run/docker.sock` 挂给 Runner 会让 CI Job 获得接近宿主机 root 的能力。只给可信项目使用，限制 protected tags，并将不可信流水线放到隔离 Runner。

十、GitHub 仓库提交与版本管理指南

为了将这份教学文档与配套资源一键保存并同步到您的 GitHub 仓库 (<https://github.com/shengl3>), 请按照以下命令行指引操作：

10.1 初始化本地 Git 仓库

```
# 切换到项目目录
cd C:\Users\A0903\gemini\antigravity\scratch\gitlab-guide

# 初始化 Git 仓库
git init

# 设置主分支名称为 main
git branch -M main

# 将所有文档与图片添加到暂存区
git add .

# 提交本地版本
git commit -m "docs: add comprehensive gitlab deployment guide and pdf"
```

10.2 关联 GitHub 远程仓库并推送

```
# 关联远程仓库地址（替换为您的具体 Repository 名称）
git remote add origin https://github.com/shengl3/gitlab-deployment-guide.git

# 推送代码至 GitHub
git push -u origin main
```

 如果使用 SSH 协议推送： `git remote set-url origin git@github.com:shengl3/gitlab-deployment-guide.git` `git push -u origin main`

文档更新：2026-08-10，基于本机 GitLab CE 19.2.1 实测。